

Les curseurs dans SQL

- Un curseur est une zone de travail privée de SQL (zone tampon)
- Il y a deux types de curseurs:
 - Curseurs implicites
 - Curseurs explicites
- Oracle utilise les curseurs implicites pour analyser et exécuter les énoncés de SQL
- Les curseurs explicites sont déclarés explicitement pas le programmeur

Les attributs des curseurs SQL

Les attributs des curseurs SQL permettent de tester les résultats des énoncés SQL

SQL%ROWCOUNT	Nombre de lignes affecté par l'énoncé SQL le plus récent (renvoie un entier).
SQL%FOUND	attribut booléen qui prend la valeur TRUE si l'énoncé SQL le plus récent affecte une ou plusieurs lignes.
SQL%NOTFOUND	attribut booléen qui prend la valeur TRUE si l'énoncé SQL le plus récent n'affecte aucune ligne.
SQL%ISOPEN	Prend toujours la valeur FALSE parce que PL/SQL ferment les curseurs implicites immédiatement après leur exécution.

Les attributs des curseurs SQL

- Supprimer de la table ITEM des lignes ayant un ordre spécifié.
Afficher le nombre de lignes supprimées.
- Exemple

```
DECLARE
    v_ ordid          NUMBER := 605;
BEGIN
    DELETE    FROM    item
    WHERE      ordid = v_ ordid;
    DBMS_OUTPUT.PUT_LINE (SQL%ROWCOUNT
                          || ' Lignes supprimées ');
END;
```

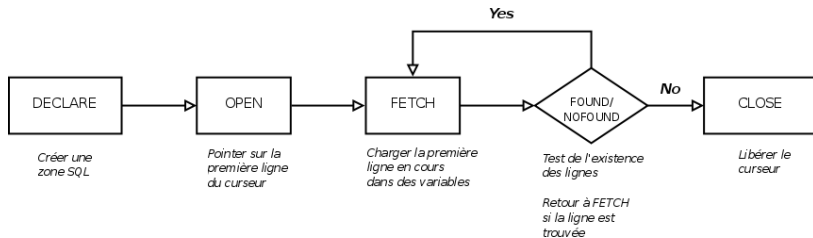
/

Les curseurs

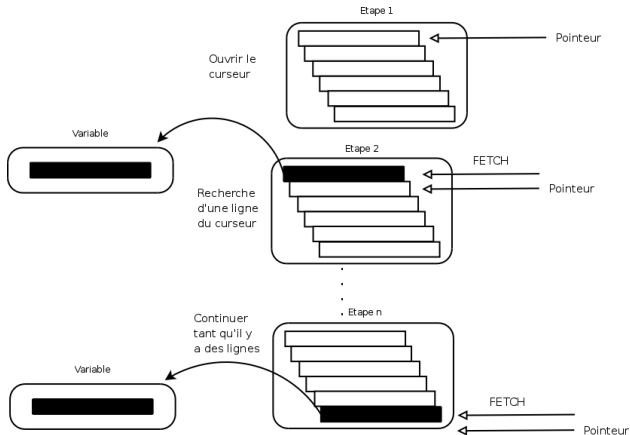
Chaque énoncé SQL exécuté par Oracle a son propre curseur :

- Curseurs implicites : déclarés pour tout énoncé SELECT du LMD ou PL/SQL
- Curseurs explicites : déclarés et nommés par le programmeur

Contrôle des curseurs explicites



Contrôle des curseurs explicites



Déclaration des curseurs

Syntaxe

CURSOR nom _du _curseur IS
 un énoncé **SELECT**;

- Ne pas inclure la clause INTO dans la déclaration du curseur
- Si le traitement des lignes doit être fait dans un ordre spécifique, on utilise la clause ORDER BY dans la requête

Déclaration des curseurs

Exemple

```
DECLARE  
CURSOR C1 IS  
    SELECT RefArt ,      NomArt , Qte Art  
    FROM Article  
    WHERE QteArt < 500;
```


Ouverture du curseur

Syntaxe

OPEN nom _du _curseur ;

- Ouvrir le curseur pour exécuter la requête et identifier l'ensemble actif
- Si la requête ne renvoie aucune ligne, aucune exception n'aura lieu
- Utiliser les attributs des curseurs pour tester le résultat du FETCH

Recherche des données dans le curseur

Syntaxe

```
FETCH      nom_du_curseur  
      INTO  [variable1 , [variable2 , ...]  
              | nom_de_record ];
```

Rechercher les informations de la ligne en cours et les mettre dans des variables.

Recherche des données dans le curseur

Exemples

```
FETCH      c1  INTO      v_RefArt , v_NomArt , v_QteArt ;
```

```
....  
OPEN      Cur_Etud ;  
LOOP  
    FETCH Cur_Etud INTO      Rec_Etud ;  
    {traitements des données recherchées}  
    ...  
END LOOP;  
...
```

Fermeture du curseur

Syntaxe

CLOSE nom _du _curseur ;

- Fermer le curseur après la fin du traitement des lignes
- Rouvrir le curseur si nécessaire
- On ne peut pas rechercher des informations dans un curseur si ce dernier est fermé

Les attributs du curseur explicite

Obtenir les informations d'état du curseur (CUR_EXP)

Attribut	Type	Description
CUR_EXP%ISOPEN	BOOLEAN	Prend la valeur TRUE si le curseur est ouvert
CUR_EXP%NOTFOUND	BOOLEAN	Prend la valeur TRUE si le FETCH le plus récent ne retourne aucune ligne
CUR_EXP%FOUND	BOOLEAN	Prend la valeur TRUE si le FETCH le plus récent retourne une ligne
CUR_EXP%ROWCOUNT	NUMBER	Retourne le nombre de lignes traitées jusqu'ici

Contrôle des recherches multiples

- Traitement de plusieurs lignes d'un curseurs en utilisant une boucle
- Rechercher une seule ligne à chaque itération
- Utiliser les attributs du curseur explicite pour tester le succès de chaque FETCH

Attribut %ISOPEN

- La recherche des lignes n'est possible que si le curseur est ouvert
- Utiliser l'attribut %ISOPEN avant un FETCH pour tester si le curseur est ouvert ou non

Exemple :

```
IF NOT      C1%ISOPEN      THEN
      OPEN      C1
END IF ;
LOOP
FETCH C1  ...
```

Attributs %FOUND, %NOTFOUND et %ROWCOUNT

- Utiliser l'attribut %ROWCOUNT pour fournir le nombre exact des lignes traitées
- Utiliser les attributs %FOUND et %NOT FOUND pour formuler le test d'arrêt de la boucle

Attributs %FOUND, %NOTFOUND et NOTFOUND%ROWCOUNT

Exemple

```

LOOP
  FETCH curs1      INTO      v_etudid , v_nom ;
  IF      curs1%ROWCOUNT > 20      THEN
    ...
  EXIT      WHEN      curs1%NOTFOUND;
  ...
END LOOP;

```

Obtenir les informations d'état du curseur

Exemple complet

DECLARE

```
nom emp.ename%TYPE;  
salaire emp.sal%TYPE;  
CURSOR C1 IS SELECT ename, NVL(sal,0) FROM emp;
```

BEGIN

```
OPEN C1;  
LOOP  
    FETCH C1 INTO nom, salaire;  
    EXIT WHEN C1%NOTFOUND;  
    DBMS_OUTPUT.PUT_LINE ( 'nom|| ' gagne ' ||  
                           salaire || ' dollars');  
END LOOP;
```

```
CLOSE C1;
```

END;

Les curseurs et les Records

Traitement des lignes de l'ensemble actif par l'affectation des valeurs à des records PL/SQL.

Exemple

```

...
CURSOR      Etud _Curs      IS
      SELECT      etudno , nom, age , ard
      FROM        etud
      WHERE       age < 26;
Etud _Record      Etud _Curs%ROWTYPE;
BEGIN
      OPEN        Etud _Curs ;
      ...
      FETCH       Etud _Curs INTO      Etud _Record ;
      ...
END;
```

Les boucles FOR des curseurs

Syntaxe

```
FOR      nom _record      IN      nom _curseur      LOOP
    -- traitement des informations
    -- utiliser des ordres SQL
    -- utiliser des ordres PL / SQL
END LOOP;
...
```

- Un raccourci pour le traitement des curseurs explicites
- OPEN, FETCH et CLOSE se font de façon implicite
- Ne pas déclarer le record, il est déclaré implicitement

Les boucles FOR des curseurs

Exemple

DECLARE

CURSOR Cur_Etud **IS**

SELECT * **FROM** Etud ;

BEGIN

FOR Rec_Etud **IN** Cur_Etud **LOOP**

DBMS_OUTPUT.PUT_LINE(Rec_Etud.etudid || '
'|| Rec_Etud.nom || ' ' || Rec_Etud.adr);

END LOOP;

END;

/

Manipulation des exceptions en PL/SQL

- Le traitement des exceptions PL/SQL : mécanisme pour manipuler les erreurs rencontrées lors de l'exécution
- Possibilité de continuer l'exécution si l'erreur n'est pas suffisamment importante pour produire la terminaison de la procédure
- Décision de continuer une procédure après erreur : décision que le développeur doit faire en fonction des erreurs possibles

Types des exceptions

- Déclenchées implicitement
 - Exceptions Oracle prédéfinies
 - Exceptions Oracle Non-prédéfinies
- Déclenchées explicitement
 - Exceptions définies par l'utilisateur

Exceptions Oracle Prédéfinies

Ce sont des erreurs Oracle courantes qui possèdent déjà un **nom** en PL/SQL. Elles sont automatiquement déclenchées par le système lorsqu'une erreur spécifique se produit.

- **Déclaration** : Elles sont automatiquement déclarées par Oracle (inutile de les déclarer dans la section DECLARE).
- **Déclenchement** : Implicite (automatique).
- **Traitement** : On les attrape par leur nom dans la section EXCEPTION.

Exemples courants :

- **NO_DATA_FOUND** : Une requête `SELECT ... INTO` ne retourne aucune ligne.
- **TOO_MANY_ROWS** : Une requête `SELECT ... INTO` retourne plus d'une ligne.
- **DUP_VAL_ON_INDEX** : Tentative d'insérer une valeur en double dans une colonne avec contrainte d'unicité (clé primaire).
- **ZERO_DIVIDE** : Tentative de division par zéro.
- **INVALID_CURSOR** : Tentative d'utiliser un curseur qui n'existe pas.

```
BEGIN
    SELECT nom INTO v_nom FROM employes WHERE id = 999; -- Supposons que 999 n'existe pas
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Erreur : Aucun employé trouvé.');
```

When the code is executed, it will attempt to select the name of an employee with ID 999. Since this ID does not exist, the NO_DATA_FOUND exception will be raised, and the message "Erreur : Aucun employé trouvé." will be printed to the output.

```
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Erreur : Trop de lignes retournées.');
```

If there were multiple employees with the same ID, the TOO_MANY_ROWS exception would be raised, and the message "Erreur : Trop de lignes retournées." would be printed.

```
END;
```

Exceptions Oracle Non-prédéfinies

Ce sont des erreurs Oracle qui ont un **numéro (code SQL)** mais pas de **nom** associé en PL/SQL. Par exemple, l'erreur ORA-02292 (violation d'intégrité référentielle - "enfant trouvé") n'a pas de nom prédéfini comme NO_DATA_FOUND.

Pour rendre le code plus lisible et traiter ces erreurs spécifiquement, le programmeur doit lui-même **donner un nom** à cette exception et l'**associer** à son numéro d'erreur Oracle.

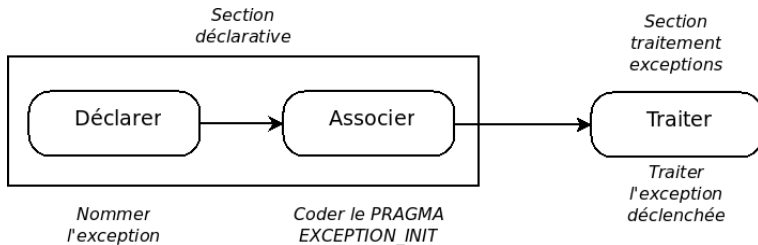
Déclaration : Obligatoire dans la section DECLARE (en tant qu'exception).

Association : Utilisation de la directive PRAGMA EXCEPTION_INIT pour lier le nom déclaré au numéro d'erreur Oracle.

Déclenchement : Implicite (automatique quand l'erreur survient).

Traitement : On les attrape par le nom qu'on leur a donné.

Capture des exceptions définies par l'utilisateur



```
DECLARE
  -- 1. Je déclare un nom pour mon exception
  ex_enfant_present EXCEPTION;
  -- 2. J'associe ce nom au code erreur Oracle -2292 (PRAGMA)
  PRAGMA EXCEPTION_INIT(ex_enfant_present, -2292);
BEGIN
  -- Tentative de suppression d'un département qui a encore des employés
  DELETE FROM departements WHERE id_departement = 10;
EXCEPTION
  -- 3. Je capture l'erreur par le nom que j'ai créé
  WHEN ex_enfant_present THEN
    DBMS_OUTPUT.PUT_LINE('Erreur : Impossible de supprimer ce département car des employés y sont rattachés.');
```

```
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Autre erreur : ' || SQLERRM);
END;
```

Capture des exceptions

Syntaxe

EXCEPTION

WHEN exception1 [**OR** exception2 ...] **THEN**

 énoncé1;

 énoncé2;

 ...

[**WHEN** exception2 [**OR** exception4 ...] **THEN**

 énoncé3;

 énoncé4;

 ...]

[**WHEN** OTHERS **THEN**

 énoncé5;

 énoncé6;

 ...]

Capture des exceptions prédéfinies: récapitulation

- Faire référence au nom dans la partie traitement des exceptions
- Quelques exceptions prédéfinies :
 - NO_DATA_FOUND
 - TOO_MANY_ROWS
 - INVALID_CURSOR
 - ZERO_DIVIDE
 - DUP_VAL_ON_INDEX

Exceptions prédéfinies

Example

BEGIN

... EXCEPTION

```

WHEN    NO_DATA_FOUND    THEN
    énoncé1;                énoncé2;
    DBMS_OUTPUT.PUT_LINE (TO_CHAR (etudno) ||
                                'Non valide ');

WHEN    TOO_MANY_ROWS    THEN
    énoncé3;                énoncé4;
    DBMS_OUTPUT.PUT_LINE ( 'Données invalides ');

WHEN    OTHERS    THEN
    énoncé5;                énoncé6;
    DBMS_OUTPUT.PUT_LINE ( 'Autres erreurs ');

```


Capture des exceptions non-prédéfinies

Exemple

Capture de l'erreur n° 2291 (violation de la contrainte intégrité).

DECLARE

cont_integrit_viol **EXCEPTION;**

PRAGMA EXCEPTION_INIT(cont_integrit_viol , -2291);

...

BEGIN

...

EXCEPTION

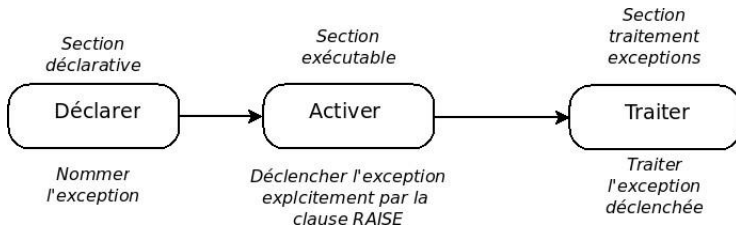
WHEN cont_integrit_viol **THEN**

DBMS_OUTPUT.PUT_LINE ('violation de contrainte
d'intégrité');

...

END;

Capture des exceptions définies par l'utilisateur



Exceptions définies par l'utilisateur

Exemple

DECLARE

x number := ...;
 x_trop_petit EXCEPTION;

 ...

BEGIN

 ...
 IF x < 5 THEN RAISE x_trop_petit;
 END IF ;

 ...

EXCEPTION

WHEN x_trop_petit THEN
 DBMS_OUTPUT.PUT_LINE('la valeur de x
 est trop petite !! ');

 ...

END;

Fonctions pour capturer les exceptions

- SQLCODE : Retourne la valeur numérique du code de l'erreur
- SQLERRM : Retourne le message associé au numéro de l'erreur

Fonctions pour capturer les exceptions

Exemple

```
...  
    v_code_erreur      NUMBER;  
    v_message_erreur   VARCHAR2(255);  
BEGIN  
  
    ...  
EXCEPTION  
  
    ...  
    WHEN OTHERS THEN  
        ...  
        v_code_erreur  :=  SQLCODE;  
        v_message_erreur :=  SQLERRM;  
        INSERT INTO erreurs VALUES ( v_code_erreur ,  
                                     v_message_erreur );  
END;
```

Les sous-programmes

- Un sous programme est une séquence d'instruction PL/SQL qui possède un nom

On distingue deux types de sous programmes :

- Les procédures
- Les fonctions

Les sous-programmes

- Une procédure : sous programme qui ne retourne des résultats seulement dans ses paramètres
- Une fonction : sous programme qui retourne des résultats dans :
 - Le nom de la fonction
 - Les paramètres de la fonction

Les procédures

Syntaxe

DECLARE

```
    ...  
    PROCEDURE <Nom_Proc >[(P1,...,Pn)] IS  
        [Déclarations locales]
```

BEGIN

```
    ...  
    EXCEPTION
```

```
    ...  
    END;
```

BEGIN

```
    /* Appel de la procédure */
```

```
    ...  
    EXCEPTION
```

```
    ...  
    END ;
```

```
    /
```


Les procédures

Syntaxe des paramètres

P1,...,Pn suivent la syntaxe :

<Nom_Arg> [**IN** | **OUT** | **IN OUT**] <Type>

Où :

- **IN** : Paramètre d'entrée
- **OUT** : Paramètre de sortie
- **IN OUT** : Paramètre d'entrée/Sortie

Par défaut le paramètre est **IN**

Mode	Rôle
IN	Valeur fournie par l'appelant, non modifiable dans la procédure.
OUT	Valeur calculée par la procédure et renvoyée à l'appelant.
IN OUT	Valeur fournie par l'appelant, modifiable, et renvoyée.

Les procédures

Exemple (utilisation de variable : IN)

DECLARE

```
PROCEDURE NouvSal (PNum IN Emp.Emp_Id %Type , PAug NUMBER ) IS  
    VSa NUMBER ( 7 , 2 );
```

BEGIN

```
    SELECT Sal INTO VSa FROM Emp
```

```
    WHERE emp_Id=PNum;
```

```
    UPDATE Emp    SET Sal = VSa+PAug WHERE Emp_Id=PNum;
```

```
    COMMIT;
```

EXCEPTION

```
    WHEN NO_DATA_FOUND THEN
```

```
        DBMS_OUTPUT.PUT_LINE( ' Employé inexistant ' );
```

END;

BEGIN

```
    NouvSal ( 7550 , 500 );
```

EXCEPTION

```
    WHEN OTHERS THEN    DBMS_OUTPUT.PUT_LINE( ' Erreur ' );
```

END ;

/

Les procédures

Exemple (utilisation de variable : OUT)

DECLARE

VErr **NUMBER** ;

PROCEDURE NouvSal(PNum Emp.Emp_Id %TYPE, PAug **NUMBER**,
PErr OUT **NUMBER**) IS

VSa **NUMBER** (7 ,2);

BEGIN

SELECT Sal **INTO** VSa **FROM** Emp **WHERE** emp_Id=PNum;
UPDATE Emp **SET** Sal = VSa+PAug **WHERE** Emp_Id=PNum;
COMMIT; PErr:=0

EXCEPTION

WHEN NO_DATA_FOUND **THEN** PErr:=1 ;

END;

Les fonctions

Syntaxe

DECLARE

[Déclarations globales]

FUNCTION <Nom_fonc>[(P1,...,Pn)] RETURN Type IS

[Déclarations locales]

BEGIN

...

RETURN valeur;

EXCEPTION

...

END;

BEGIN

-- Appel a la fonction

...

EXCEPTION

...

END ;

/

Les fonctions

Exemple

DECLARE

```
VNomComple VARCHAR2(40);  
FUNCTION NomComple(PNum Emp. Emp _Id%TYPE, PErr OUT NUMBER )  
  RETURN VARCHAR2 IS  
  VLastName Emp.Last_Name %Type;  
  VFirstName Emp.First_Name %Type;  
BEGIN  
    SELECT Last_Name , First_Name  
      INTO VLastName , VFirstName  
      WHERE Emp-Id=PNum; PErr := 0;  
  RETURN VLastName || ' ' || VFirstName;  
EXCEPTION  
  WHEN NO_DATA_FOUND THEN PErr := 1; RETURN Null;  
END ;
```

Le paramètre IN OUT

- Paramètre jouant le rôle des deux paramètres IN et OUT
- Obligatoire de le spécifier

Exemple :

```
SQL> Create or replace procedure affnom(v_nom IN OUT varchar2) IS  
  2 BEGIN  
  3   v_nom := UPPER (v_nom);  
  4 END affnom;  
  5 /  
Procédure créée .
```

Appel de affnom de SQL*Plus

- Définition d'une variable liée
- Initialisation de la variable

```
SQL> var name varchar2(10);  
SQL> begin :name := 'mark' ; end ;  
/  
Procédure PL/SQL terminée avec succès  
SQL> print name  
NAME  
-----  
mark
```

Appel de affnom de SQL*Plus

- Exécution de la procédure avec un paramètre IN OUT
- Affichage la nouvelle valeur de la variable

```
SQL> execute affnom (:name);  
Procédure PL/SQL terminée avec succès.  
SQL> print name  
NAME
```

MARK

Passage de paramètres

Il y a plusieurs façons de passage de paramètres :

- Appel de la procédure en spécifiant les paramètres
- Appel de la procédure sans paramètre si ce dernier est un paramètre d'entrée initialisé
- Appel de la procédure en changeant la position des paramètres (il faut spécifier le nom du paramètre)

Passage de paramètres

Exemple

```
CREATE OR REPLACE PROCEDURE renseign_etud  
(v_nom IN etud.nom%type default 'inconnu',  
  v_adr IN etud.adr%type default 'inconnu')  
IS  
BEGIN  
    INSERT INTO etud  
        VALUES (etud_etudid.nextval, v_nom, v_adr);  
END;
```

Passage de paramètres

Exemple

```
SQL> begin
  2  renseign_etud('mark', 'paris');
  3  renseign_etud;
  4  renseign_etud(v_adr => 'lyon');
  5  end;
  6  /
```

Procédure PL/SQL terminée avec succès.

```
SQL> select * from etud;
```

ETUDID	NOM	ADR
6	mark	paris
7	inconnu	inconnu
8	inconnu	lyon

```
SQL>
```

Les procédures et les fonctions stockées

- Sont des blocs PL/SQL qui possèdent un nom
- Consistent à ranger le block PL/SQL compilé dans la base de données (CREATE)
- Peuvent être réutilisées sans être recompilées (EXECUTE)
- Peuvent être appelées de n'importe quel bloc PL/SQL
- Peuvent être regroupées dans un package

Les procédures stockées

Syntaxe

```
CREATE [ OR REPLACE ] PROCEDURE <Nom_Proc>[(P1,...,Pn)]  
    [Déclarations des variables locales]  
BEGIN  
    ...  
EXCEPTION  
    ...  
END;  
/
```

- Procedure Created : La procédure est correcte
Ou
- Procedure Created with compilation errors : Corriger
les erreurs → **SHOW ERRORS;**

Les procédures stockées

Exemple

```
CREATE [ OR REPLACE ] PROCEDURE  
    AjoutProd ( PrefPro Prod . RefPro%TYPE, ... PPri Uni  
                Prod . PriUni% TYPE, PErr OUT Number) IS  
  
    BEGIN  
        INSERT      INTO      Prod VALUES(PrefPro , ... , PPri Uni );  
        COMMIT;  
        PErr := 0;  
  
    EXCEPTION  
        WHEN      DUP_VAL_ON_INDEX      THEN  
            PErr := 1;  
  
    END;  
  
/
```

Appel des procédures stockées

Syntaxe

La procédure stockée est appelée par les applications soit :

- En utilisant son nom dans un bloc PL/SQL (autre procédure)
Par execute dans SQL*Plus
- Dans un bloc PL/SQL :

DECLARE
BEGIN

<Nom_Procedure >[<P1 > , ... , <Pn >];

END;

- Sous SQL*PLUS :

EXECUTE <Nom_Procedure >[<P1 > , ... , <Pn >];

Appel des procédures stockées

Exemple

```
ACCEPT VRefPro ...  
ACCEPT VPriUni ...  
DECLARE  
    VErr NUMBER;  
BEGIN  
    AjoutProd(&VRefPro ,... ,&VPriUni , VErr );  
    IF VErr=0 THEN  
        DBMS_OUTPUT.PUT_LINE('Opération Effectuer');  
    ELSE DBMS_OUTPUT.PUT_LINE('Erreur');  
    END IF ;  
END ;  
/
```


Les fonctions stockées

Syntaxe

```
CREATE [ OR REPLACE ] FUNCTION <Nom_Fonc>[(P1,...,Pn)]  
  RETURN Type IS  
  [Déclarations des variables locales]  
  BEGIN  
    Instructions SQL et PL/Sql  
    RETURN(Valeur)  
  EXCEPTION  
    Traitement des exceptions  
  END;  
/
```

- function Created : La fonction est correcte
Ou
- function Created with compilation errors :
Corriger les erreurs → **SHOW ERRORS;**

Les fonctions stockées

Exemple

```
CREATE [ OR REPLACE ] FUNCTION NbEmp (PNumDep Emp.Dept_Id%Type ,  
                                           PErr Out Number) Return Number  
  
IS  
    VNb Number(4);  
  
BEGIN  
    Select Count(*) Into VNb From Emp Where Dept_Id=PNumDep;  
    PErr:=0  
    Return VNb;  
  
Exception  
    When No-Data-Found Then  
        PErr := 1;  
        Return Null;  
  
END;  
/
```

Appel des fonctions stockées

Syntaxe

La fonction stockée est appelée par les applications soit :

- Dans une expression dans un bloc PL/SQL
- Dans une expression dans par la commande EXECUTE (dans SQL*PLUS)
- Dans un bloc PL/SQL :

DECLARE
BEGIN

<var> := <Nom_fonction> [<P1>, ..., <Pn>]

END;

- Sous SQL*PLUS :

EXECUTE <var> := <Nom_fonction> [<P1>, ..., <Pn>]

Appel des fonctions stockées

Exemple

```
Accept VDep ...
```

```
Declare
```

```
    VErr Number;
```

```
    VNb Number(4);
```

```
Begin
```

```
    VNb :=NbEmp(&VDep, VErr );
```

```
    If VErr=0 Then
```

```
        DBMS_Output.Put_Line('Le nombre  
                                d'employées est : ' || VNb);
```

```
    Else
```

```
        DBMS_Output.Put_Line('Erreur');
```

```
    End If;
```

```
End;
```

```
/
```

Appel des fonctions stockées

Exemple

```
SQL> VARIABLE VNb  
SQL> EXECUTE :VNb:=NbEmp(&VDep, VErr);  
Procédure PL/SQL terminée avec succès.  
SQL> PRINT VNb
```

VNB

300

Suppression des procédures et des fonctions stockées

Syntaxe

- Comme tout objet manipulé par Oracle, les procédures et les fonctions peuvent être supprimées si nécessaire
- Cette suppression est assurée par la commande suivante :

DROP	PROCEDURE	nomprocedure ;
DROP	FUNCTION	nomfonction ;

Suppression des procédures et des fonctions stockées

Exemple

```
SQL> DROP      PROCEDURE      AjoutProd ;  
Procedure  dropped .
```

```
SQL> DROP      FUNCTION      NbEmp ;  
Function  dropped .
```

Les procédures et les fonctions stockées

Quelques commandes utiles :

SELECT object name , object type **from** obj ;

DESC nomprocedure

DESC nomfonction

Remarque :

DESC table	SQL*Plus, SQL Developer (fenêtre SQL)	Commande client	DESC employees;
SELECT FROM user_tab_columns	Partout (SQL, PL/SQL)	SQL	SELECT * FROM user_tab_columns WHERE table_name='EMPLOYEES';
DBMS_METADATA.GET_DDL	Partout (surtout PL/SQL)	Procédure stockée	SELECT DBMS_METADATA.GET_DDL('TABLE','EMPLOYEE S') FROM dual;

Les packages

- Un objet PL/SQL qui stocke d'autres types d'objet :
procédures, fonctions, curseurs, variables, ...
- Consiste en deux parties :
 - Spécification (déclaration)
 - Corps (implémentation)
- Ne peut pas être appelé, ni paramétré ni imbriqué
- Permet à Oracle de lire plusieurs objets à la fois en mémoire

Développement des packages

- Sauvegarder l'énoncé de CREATE PACKAGE dans deux fichiers différents (ancienne/dernière version) pour faciliter de éventuelles modifications
- Le corps du package ne peut pas être compilé s'il n'est pas déclaré (spécifié)
- Restreindre les privilèges pour les procédures à une personne donnée au lieu de lui donner tout les droits sur toutes les procédures

GRANT EXECUTE ON mon_package **TO** utilisateur;

La spécification du package

Syntaxe

Contient la déclaration des curseurs, variables, types, procédures, fonctions et exceptions

```
CREATE [OR REPLACE] PACKAGE <Nom_Package>  
    IS [Déclaration des variables et Types]  
        [Déclaration des curseurs]  
        [Déclaration des procédures et fonctions]  
        [Déclaration des exceptions]  
END[<Nom_Package >];  
/
```

La spécification du package

Exemple

```
Create Or Replace Package PackProd  
  Is Cursor CProd Is Select RefPro , DesPro  
    From Produit ;  
  Procedure  AjoutProd ( PrefPro Prod . Ref Pro%Type ,  
                        ... , PErr Out Number ) ;  
  Procedure  ModifProd ( PrefPro Prod . Ref Pro%Type ,  
                        ... , PErr Out Number ) ;  
  Procedure  SuppProd ( PrefPro Prod . RefPro%Type ,  
                        ... , PErr Out Number ) ;  
  Procedure  AffProd ;  
EndPackProd ;  
/
```

Le corps du package

Syntaxe

On implémente les procédures et fonctions déclarées dans la spécification

```
Create [Or Replace] Package Body <Nom_Package>      Is  
      [Implémentation procédures | fonctions]  
End [<Nom_Package >];  
/
```

Le corps du package

Syntaxe

```
Create Or Replace Package Body PackProd
Is
  Procedure AjoutProd ( PrefPro Prod . RefPro%Type ,
                        ... , PErr Out Number)
Is
Begin
    Insert Into Prod Values(PrefPro , ... , PPri Uni );
    Commit;
    PErr := 0;
Exception
    When Dup_Val_On_Index Then    PErr := 1;
    When Others Then    PErr := 1;
End ;
```

Le corps du package

Syntaxe

```
Procedure ModifProd ( PrefPro Prod . RefPro%Type ,  
                      ... , PErr Out Number)  
    Is B Boolean ;  
Begin  
    ...  
EndPackProd ;  
/
```

```
CREATE OR REPLACE PACKAGE BODY gestion_employes AS
-- MAINTENANT on écrit le vrai code
PROCEDURE augmenter_salaire(p_id NUMBER, p_pourcentage NUMBER) IS
BEGIN
    UPDATE employes SET salaire = salaire * (1 + p_pourcentage/100)
    WHERE id = p_id;
END augmenter_salaire;

FUNCTION calculer_prime(p_id NUMBER) RETURN NUMBER IS
    v_prime NUMBER;
BEGIN
    SELECT salaire * 0.1 INTO v_prime FROM employes WHERE id = p_id;
    RETURN v_prime;
END calculer_prime;
END gestion_employes;
/
```


Appels des procédures / fonctions du package

Syntaxe

Les procédures et les fonctions définies dans un package sont appelées de la façon suivante :

`<NomPackage>.<NomProcedure >[( Paramètres ) ];`

`Var := <NomPackage>.<NomFonction >[( Paramètres ) ];`

Appels des procédures / fonctions du package

Exemple

```
Accept VRef Prompt ' ..... ' ;  
Accept VPri Prompt ' ..... ' ;  
Declare  
    VErr Number ;  
Begin  
    PackProd . ModifProd (&VRef , ... , &VPri , VErr ) ;  
    If VErr = 0 Then  
        DBMS_Output . Put_Line ( ' Traitement effectué ' ) ;  
    Else  
        DBMS_Output . Put_Line ( ' Erreur ' ) ;  
    End If ;  
End ;  
/
```

Packages : Exemples

Création le corps du package suivant en mode interactif :

```
SQL> create or replace package body pack1 is
      2 function double_x(x number) return number is
      3 begin
      4 return(2*x);
      5 end;
      6 end;
      7 /
```

Avertissement : Corps de package créé avec erreurs de compilation.

Packages : Exemples

Pour afficher les erreurs on utilise la commande SHOW ERRORS

```
SQL> show errors
```

Erreurs pour PACKAGE BODY PACK1 :

```
LINE/COL  ERROR
```

```
-----  
0/0       PL/SQL: Compilation unit analysis terminated  
1/14      PLS-00201: l'identificateur 'PACK1' doit être déclaré  
1/14      PLS-00304: impossible de compiler le corps de 'PACK1' sans  
           spécification
```

```
SQL>
```

Les triggers (Déclencheurs)

- Un trigger est un programme PL/SQL qui s'exécute automatiquement avant ou après une opération LMD (Insert, Update, Delete)
- Contrairement aux procédures, un trigger est déclenché automatiquement suite à un ordre LMD

Événement-Condition-Action

- Un trigger est activé par un événement
 - Insertion, suppression ou modification sur une table
- Si le trigger est activé, une condition est évaluée
 - Prédicat qui doit retourner vrai
- Si la condition est vraie, l'action est exécutée
 - Insertion, suppression ou modification de la base de données

Composants du trigger

À quel moment se déclenche le trigger ?

- BEFORE : le code dans le corps du triggers s'exécute avant les évènements de déclenchement LMD
- AFTER : le code dans le corps du triggers s'exécute avant les évènements de déclenchement LMD

Composants du trigger

Les évènements du déclenchement :

- Quelles sont les opérations LMD qui causent l'exécution du trigger ?
 - INSERT
 - UPDATE
 - DELETE
 - La combinaison des ces opérations

Composants du trigger

Le corps du trigger est défini par un bloc PL/SQL anonyme

```
[DECLARE]  
BEGIN  
[ EXEPTION ]  
END;
```

Composants du trigger

Syntaxe

```
Create [Or Replace] Trigger <Nom_Trigger >  
  [Before | After] <Opération DML> On <Nom_Table>  
  [For Each Row][When <Condition >]  
  Declare  
  Begin  
  Exception  
  End ;  
  /
```

Composants du trigger

Exemple

*Création d'un trigger qui remplit la table statistique
(Nom_Table → Nb_Insert) lors d'une insertion dans la table facture*

```
Create Or Replace Trigger StartFacture  
  After Insert On Facture  
  For Each Row  
  
Declare  
  VNbInsert Number;  
  
Begin  
  Select Nb_Insert Into VNbInsert  
    From Statistique Where Nom_Table='Facture';  
  Update Statistique  
    Set Nb_Insert = VNbInsert+1  
    Where Nom_Table='Facture';  
  
Exception  
  When No_Data_Found Then  
    Insert Into Statistique Values (1,'Facture');  
  
End;  
/
```

Manipulation des triggers

- Activer ou désactiver un Trigger:

Alter Trigger <Nom _Trigger > [Enable | Disable];

- Supprimer un Trigger:

Drop Trigger <Nom _Trigger >;

- Déterminer les triggers de votreBD:

Select Trigger _Name **From** User _Triggers

Les attributs :Old et :New

Ces deux attributs permettent de gérer l'ancienne et la nouvelle valeurs manipulées

- Insert(...) ... → New
- Delete ...
- Where(...) ... → Old
- Update ...
- Set (...) ... → New
- Where(...) ... → Old

Les attributs :Old et :New

Exemple :

Etudiant (Matr _Etu , Nom , ... , Cod _Cla)

Classe (Cod _Cla , Nbr _Etu)

Trigger mettant à jour la table classe suite à une insertion d'un nouvel étudiant

```
Create or Replace Trigger MajNbEtud  
  After Insert On Etudiant  
  For Each Row
```

```
Begin
```

```
  Update Classe
```

```
    Set Nbr _Etud = Nbr _Etud+1
```

```
    Where Cod _Cla =:New.Cod _Cla ;
```

```
End ;
```

```
/
```

Les prédicats inserting, updating et deleting

- Inserting:
 - True: Le trigger est dédenché suite à une insertion
 - False: Sinon
- Updating:
 - True: le trigger est dédenché suite à une mise à jour
 - False: sinon
- Deleting:
 - True: le trigger est dédenché suite à une suppression
 - False: sinon

Les prédicats inserting, updating et deleting

Exemple

```
Create Or Replace Trigger MajNbEtud
  After Insert Or Delete On Etudiant
  For Each Row
Begin
  If Inserting Then
    Update Classe Set Nbr_Etud = Nbr_Etud+1
    Where Cod_Cla =:New.Cod_Cla ;
  End If ;
  If Deleting Then
    Update Classe Set Nbr_Etud = Nbr_Etud -1
    Where Cod_Cla =:Old.Cod_Cla ;
  End If ;
End ;
/
```


Séquences

Syntaxe

Auto-incrémentation d'une colonne (évite les doublons)

- Définition :

```
CREATE SEQUENCE sequence_name  
  [INCREMENT BY #]  
  [START WITH #]  
  [ MAXVALUE # | NOMAXVALUE]  
  [ MINVALUE # | NOMINVALUE]  
  [CYCLE | NOCYCLE]
```

- Suppression

```
DROP SEQUENCE sequence_name
```

- Pseudo-colonne CURRVAL : Valeur courante de la séquence
- Pseudo-colonne NEXTVAL : Incrémentation de la séquence et retourne la nouvelle valeur

Séquences

Exemple

```
drop sequence SEQ_ANNOTATION;  
create sequence SEQ_ANNOTATION  
            start with 1  
            increment by 1  
            nocycle  
            maxvalue 30000
```

```
INSERT INTO prof (prof_num, prof_nom, prof_prenom)  
VALUES (SEQ_ANNOTATION.NEXTVAL, 'Dupond' 'Michel')
```

```
SELECT seq_annotation.CURRVAL from dual;
```

SQL dynamique

- Construction dans un programme une requête SQL avant de l'exécuter
- Possibilité de création d'un code générique et réutilisable (sinon simple paramétrage de valeur de remplacement de la clause where)
- `Execute immediate chaine_de_caract`eres ;`
`chaine_de_caract`eres` est une commande sql donnée entre '`...`'

SQL dynamique

- Exemple 1 :

Begin

Execute immediate 'create table test (col1 :number);';
End;

- Exemple 2 :

Declare

w_req **varchar2**(4000);

Begin

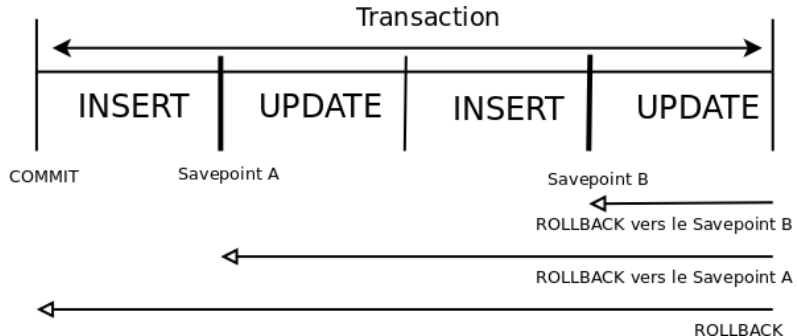
w_req := 'create table test (col1 :number);';
Execute immediate w_req;
End;

Contrôle de transactions

Commandes COMMIT et ROLLBACK

- Lancer une transaction avec la première commande du LMD à la suite d'un COMMIT ou un ROLLBACK
- Utiliser le COMMIT ou le ROLLBACK de SQL pour terminer une transaction

Commande ROLLBACK



Contrôle de transactions

Déterminer le traitement des transactions pour le bloc PL/SQL suivant

```
BEGIN
    INSERT INTO temp(num_col1 , num_col2 , char_col)
        VALUES (1, 1, 'ROW 1');
    SAVEPOINT a;
    INSERT INTO temp(num_col1 , num_col2 , char_col)
        VALUES (2, 2, 'ROW 2');
    SAVEPOINT b;
    INSERT INTO temp(num_col1 , num_col2 , char_col)
        VALUES (3, 3, 'ROW 3');
    SAVEPOINT c;
    ROLLBACK TO SAVEPOINT b;
    COMMIT;
END;
```

```
CREATE TABLE employes (
    id      NUMBER PRIMARY KEY,
    nom     VARCHAR2(50),
    salaire NUMBER
);
```

```
CREATE TABLE log_employes (
    id_log  NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    action  VARCHAR2(20),
    date_log DATE DEFAULT SYSDATE,
    details VARCHAR2(100)
);
```

```
CREATE OR REPLACE TRIGGER verif_salaire_before_insert
    BEFORE INSERT ON employes
    FOR EACH ROW
BEGIN
    IF :NEW.salaire <= 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Le salaire doit être positif.');
```

```
    END IF;
```

```
END;
```

```

/
```

```
CREATE OR REPLACE TRIGGER log_employes_after_insert
    AFTER INSERT ON employes
    FOR EACH ROW
BEGIN
    INSERT INTO log_employes (action, details)
    VALUES ('INSERT', 'Nouvel employé : ' || :NEW.nom || ', salaire=' || :NEW.salaire);
END;
```

```
-- Insertion valide
```

```
INSERT INTO employes (id, nom, salaire) VALUES (1, 'Alice', 3500);
```

```
-- Résultat : insertion réussie, une ligne apparaît dans log_employes.
```

```
-- Insertion avec salaire nul (<=0)
```

```
INSERT INTO employes (id, nom, salaire) VALUES (2, 'Bob', 0);
```

```
-- Résultat : erreur ORA-20001 : Le salaire doit être positif. Aucune insertion.
```

```
-- Vérification de la table de log
```

```
SELECT * FROM log_employes;
```

```
-- On doit voir une ligne pour Alice.
```